

EHAKBANG · INTERNAL DOCUMENTATION

eGov API Reference

All external eGov/eGovPay/eMessage/Liveness endpoints used by the app,
how each is wired in, and why it exists

Generated 2026-07-27 · Source: eGovAI-apidoc.md, eGovPay-apidoc.md, and lib/server/*.ts

Contents

1. Overview & Architecture	
2. eGov AI Assistant — used for journey generation	
3. eMessage — SMS Push (Push SMS)	
4. eGovPay — Payment Gateway	
4.1 Generate Payment	
4.2 Check Transaction Details	
4.3 Void Transaction (unused)	
5. eGov Face Liveness API	
6. eHakbang's Own Routes (what the frontend calls)	
7. Environment Variables Reference	

1. Overview & Architecture

eHakbang integrates four external services provided for the eGov hackathon/sandbox: the **eGov AI Assistant platform** (AI text generation + SMS), the **eGovPay** payment gateway, and the **eGov Face Liveness API**. All credentials for these are server-only environment variables — no key ever reaches the browser. The browser only ever talks to eHakbang's own Next.js API routes under `app/api/`, which act as authenticated proxies to the real eGov services.



Client wrapper	External service	Used by eHakbang route(s)
<code>lib/server/egov-ai.ts</code>	eGov AI Assistant	<code>/api/journey</code>
<code>lib/server/emessage.ts</code>	eMessage (Push SMS)	<code>/api/notifications/auto-apply</code>
<code>lib/server/egovpay.ts</code>	eGovPay	<code>/api/payment</code> , <code>/api/payment/[uuid]</code> , <code>/api/payment/webhook</code>
<code>lib/server/liveness.ts</code>	eGov Face Liveness	<code>/api/liveness/session</code> , <code>/api/liveness/result/[token]</code> , also called again inside <code>/api/payment</code>

2. eGov AI Assistant

Base: `{{base}}/api/v1/egov/integration/...` — a hackathon-issued sandbox. eHakbang uses only this one AI endpoint from the platform; authenticated with a bearer token provisioned ahead of time (see §7).

AI Assistant — powers journey generation

POST `{{base}}/api/v1/egov/integration/ai_assistant/generate`

Field	Type	Required	Description
<code>prompt</code>	string	Yes	Free-text natural-language question.
<code>category</code>	string	Yes	Region/category code, e.g. <code>"PH"</code> .

```
// Response 200
{
  "data": "To obtain your digital Taxpayer Identification Number (TIN) ID...",
  "session_id": "b67017a4-da57-40ab-96c9-ca0ccb530ec7"
}
```

Purpose: a general-purpose conversational AI endpoint scoped to Philippine government services — takes one free-text prompt, returns one free-text answer (no structured-output / JSON-schema mode).

Used in eHakbang? Yes — this is the engine behind **journey generation**.

How: `lib/server/egov-ai.ts` → `generateJourneyWithEgovAi()` builds one large prompt that:

- Instructs the model to act as "eHakbang's requirements assistant," listing the exact steps a citizen must take across SSS, PhilHealth, Pag-IBIG, BIR, PSA, DOLE, GSIS, DSWD, LGU/barangay, COMELEC, etc. for a given life event
- Embeds a strict JSON-shape specification in the prompt itself (since the endpoint has no schema-enforcement feature) — asking for `summary` + an array of `steps` with fields like `agency_name`, `documents_required`, `fee`, `required_fields`, `egov_search_term`, etc.
- Sends `category: "PH"` and the built prompt as `prompt`

The response's `data` string is parsed defensively (code-fence stripped, `JSON.parse` 'd, shape-validated) into a `GeneratedJourney`. A post-processing step (`inferFulfillsId`) then maps each step to a known ID type (TIN, SSS, PhilHealth, Pag-IBIG, PhilSys) since the model doesn't reliably tag this itself. The result is consumed by `lib/server/journey-requirements.ts`, cached, and served through the app's own `POST /api/journey` route (see §6).

3. eMessage — SMS Push

POST `{{base_url}}/messaging/v1/sms/push`

Field	Type	Required	Description
<code>number</code>	string	Yes	Recipient mobile number, E.164 format (e.g. <code>+639090000000</code>).
<code>message</code>	string	Yes	SMS body text.

Authenticated via header `X-EMESSAGE-Auth: {{api_token}}` .

```
// Response 201
{ "data": { "message": "SMS was successfully created." } }
```

Purpose: sends a one-way SMS notification to a phone number.

Used in eHakbang? Yes.

How: `lib/server/emessage.ts` → `pushSms()` wraps this call, authenticating with `EMESSAGE_API_TOKEN` / `EMESSAGE_BASE_URL`. It's invoked from `POST /api/notifications/auto-apply` (see §6) to text the citizen once their "auto-apply" submission has gone out to all agencies — e.g. *"eHakbang: Your [event] application was successfully submitted to all agencies."*

Note: because eHakbang currently captures zero personal data (no phone-number field yet), the route sends every notification to one fixed placeholder test number (`TEMP_TEST_RECIPIENT`) rather than a real per-citizen number — a known stand-in until phone capture is added.

4. eGovPay — Payment Gateway

A single integrated payment gateway for generating hosted payment links, checking transaction status, and voiding transactions. Authenticated via header `X-eGovPay-Token` (a `test_`-prefixed token runs in sandbox mode, never touching live rails).

4.1 Generate Payment

POST

{{base_url}}/api/v1/transaction

Field	Type	Required	Description
items[]	array	Yes	Line items (name + amount).
amount	double	Yes	Total transaction amount.
settlement_template_uuid	uuid	Yes	Which bank-settlement template to use.
redirect_url	url	Yes	Where the customer lands after paying.
txnid	string	Yes	Merchant-generated transaction ID.
callback_url	url	Yes	Webhook notified on every status change.
digest	string	Yes	hash_hmac('sha256', "\$amount \$txnid", \$token)
currency , mobile , email , name , expires_at , link_expires_at , description	—	No	Optional metadata / expiry controls.

```
// Response 201
{
  "data": {
    "uuid": "a23977c3-f2f2-4e5c-bf53-94bcff48e49c",
    "url": "https://egovpay-pgi-dev.oueg.info/a23977c3-...",
    "channel": { "refno": "0IOKUXQ5XX" }
  }
}
```

Purpose: creates a payment transaction and returns a hosted checkout link the citizen is redirected to in order to pay.

Used in eHakbang? Yes — this is the core of the **pay-for-fees** flow.

How: `lib/server/egovpay.ts` → `createTransaction()` computes the required HMAC `digest` (with a quirk noted in code: the HMAC key strips the `test_` prefix even though the header keeps it, and the amount must be sent as a minimal numeric string, not forced to 2 decimals). It's called from `POST /api/payment` (see §6), which re-

derives the fee-bearing steps from the canonical journey server-side (never trusting client-submitted amounts) and requires a verified face-liveness token before starting payment.

4.2 Check Transaction Details

GET `{{base_url}}/api/v1/transaction/{{transaction_uuid}}`

```
// Response 200 (abridged)
{
  "data": {
    "uuid": "...", "refno": "0IOKUXQ5XX", "txnid": "TESTREF123",
    "payment_status": "INITIAL", "paid_at": null,
    "amount": "1000.0000", "currency": "PHP", ...
  }
}
```

Purpose: looks up the current status/details of a transaction by UUID.

Used in eHakbang? Yes.

How: `lib/server/egovpay.ts` → `getTransaction()` , exposed via `GET /api/payment/[uuid]` . The route derives a simple boolean `paid` from `paid_at != null` — treated as the one unambiguous signal — rather than trusting the `payment_status` string, and this is what the `/journey/pay/callback` page polls to confirm payment completed.

4.3 Void Transaction

PUT `{{base_url}}/api/v1/transaction/{{transaction_uuid}}/void`

Purpose: cancels/voids an existing transaction by UUID.

Used in eHakbang? Implemented as `voidTransaction()` in `lib/server/egovpay.ts` , but not yet called by any route — available for a future "cancel payment" feature.

Webhook note: eGovPay requires every transaction to carry a reachable `callback_url` . eHakbang points this at its own `POST /api/payment/webhook` , which currently just logs the payload and acks — it is *not* the source of truth for the UI (polling `GET /api/payment/[uuid]` is), and the eGovPay docs don't yet specify a signature scheme to verify the webhook is genuine, so no verification is implemented.

5. eGov Face Liveness API

Not included in the two apidoc source files, but referenced throughout the payment/verification flow and implemented in `lib/server/liveness.ts`. Authenticated via header `x-api-key`.

5.1 Create Liveness Session

POST `{{base_url}}/v1/liveness/session`

Field	Type	Required	Description
<code>action</code>	string	Yes	<code>"redirect"</code> — hosted capture flow.
<code>callback_url</code>	string	Yes	Where the citizen is sent back to after capture.
<code>delay</code>	number	No	Delay before redirect (ms), default 3000.

Purpose: starts a face-liveness capture session and returns a hosted URL + session token.

Used in eHakbang? Yes — proxied by `POST /api/liveness/session` (see §6), which sends the citizen to eGov's hosted capture page before sensitive actions (paying fees, submitting an application).

5.2 Get Liveness Result

GET `{{base_url}}/v1/liveness/result/{{token}}`

```
// Response 200
{ "status": "SUCCEEDED", "confidence_score": 99.2, "reference_image_url": "..." }
```

Purpose: retrieves the outcome of a previously created liveness session.

Used in eHakbang? Yes — `lib/server/liveness.ts` → `getLivenessResult()` applies the recommended security thresholds itself: `verified = status === "SUCCEEDED" && confidence_score >= 95.0`. Exposed to the client via `GET /api/liveness/result/[token]`, and also called *server-side again* inside `POST /api/payment` — a client-only "verified" flag is bypassable by hitting the payment route directly, so the check is independently re-verified before any transaction is created.

Dev mock: when `LIVENESS MOCK=true` (or the API isn't configured) and `NODE_ENV !== "production"`, the wrapper auto-approves every check and `/api/liveness/session` redirects to an in-app mock capture page (`/dev/liveness-mock`) instead of the real eGov flow. This path is hard-disabled in production regardless of env vars.

6. eHakbang's Own Routes

These are the only endpoints the browser ever calls. Each one is a thin, validating proxy in front of the external services above.

Route	Purpose	Calls out to
POST <code>/api/journey</code>	Returns AI-generated (or cached, or seed-fallback) requirement steps for a predefined life event. Manual-refresh path; the main page prefetches server-side instead.	eGov AI Assistant (§2), via <code>getOrRegenerateJourney()</code>
POST <code>/api/payment</code>	Starts an eGovPay transaction for the fee-bearing steps the citizen selected. Re-derives fees server-side (never trusts client amounts) and requires a verified liveness token.	eGov AI Assistant (re-derive journey), eGov Face Liveness (re-verify), eGovPay Generate Payment (§4.1)
GET <code>/api/payment/[uuid]</code>	Polls a transaction's paid status; derives <code>paid</code> from <code>paid_at</code> .	eGovPay Check Transaction (§4.2)
POST <code>/api/payment/webhook</code>	Reachable callback endpoint eGovPay requires on every transaction; logs and acks. Not the UI's source of truth.	(inbound only — receives eGovPay's notification)
POST <code>/api/liveness/session</code>	Creates a face-liveness capture session and returns the hosted capture URL + token (or the dev mock equivalent).	eGov Face Liveness Session (§5.1)
GET <code>/api/liveness/result/[token]</code>	Fetches and evaluates a liveness session's result.	eGov Face Liveness Result (§5.2)
POST <code>/api/notifications/auto-apply</code>	Texts the citizen once an auto-apply submission finishes (currently to a fixed placeholder number, pending real phone capture).	eMessage Push SMS (§3)

7. Environment Variables Reference

Variable	Used by	Purpose
EGOV_AI_BASE_URL	egov-ai.ts	Base URL for the eGov AI Assistant platform.
EGOV_AI_API_TOKEN	egov-ai.ts	Bearer token for the AI Assistant endpoint (\$2), provisioned ahead of time.
EMESSAGE_BASE_URL	emessage.ts	Base URL for the eMessage SMS service.
EMESSAGE_API_TOKEN	emessage.ts	X-EMESSAGE-Auth token.
EGOVPAY_BASE_URL	egovpay.ts	Base URL for the eGovPay gateway.
EGOVPAY_API_TOKEN	egovpay.ts	X-eGovPay-Token ; test_ prefix = sandbox mode.
EGOVPAY_SETTLEMENT_TEMPLATE_UUID	egovpay.ts	settlement_template_uuid for all transactions.
EGOV_LIVENESS_BASE_URL	liveness.ts	Base URL for the eGov Face Liveness API.
EGOV_LIVENESS_API_KEY	liveness.ts	x-api-key for liveness session/result calls.
LIVENESS MOCK	liveness.ts	Dev-only flag to force-mock liveness (never active in production).

eHakbang — internal engineering reference · compiled from eGovAI-apidoc.md, eGovPay-apidoc.md, and lib/server/*.ts